Pet Store API TESTING

NAME: BHUMIKA

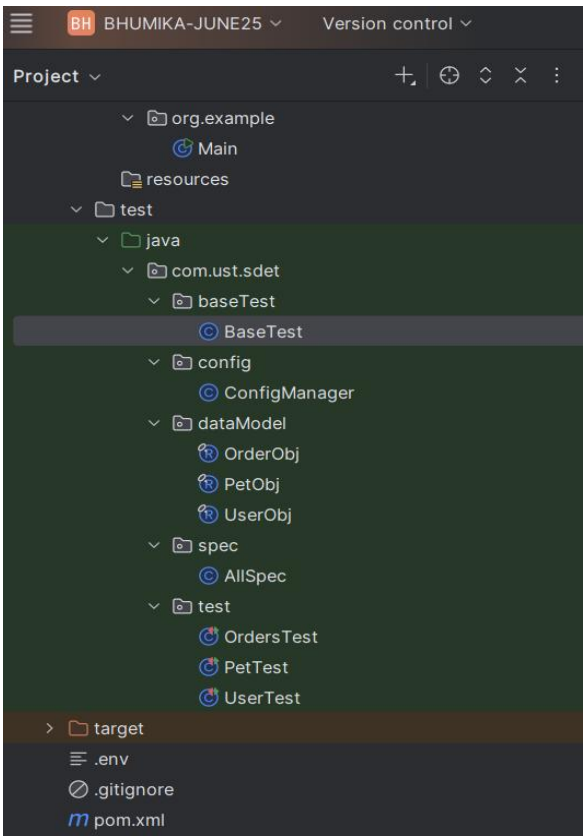
Objective

I have Wrote an API automation framework for the Swagger PetStore application using Rest Assured and JUnit 5. The main purpose of this framework is to validate the Pet, Store, and User APIs by performing different CRUD operations such as Create, Get, Update, and Delete.

Project Structure

The project is divided into different packages to keep the code clean and easy to maintain.

Folder Structure



Technology Stack

The following technologies I have used for implementation:

Technology	Purpose
Java	Programming Language
Rest Assured	API Automation
JUnit 5	Test Execution
Maven	Build Management
Git	Version Control

basetest

This package contains the BaseTest class. The base URL is initialized here so that it can be used across all test classes.

And I have wrote in that base Test Request Builder so that I don't need to pass manually every time.

config

The ConfigManager class stores the application URL,
which is coming from .env file

Data model

This package contains the request payload classes.

- PetObj
- OrderObj
- UserObj

I have used Java records to create these objects. So that no need to write getters and setters.

spec

The AllSpec class contains common response validations such as:

- Success response (200)
- Not found (404)

This avoids writing the same validations in every test.

test

This package contains all the API test classes.

- PetTest
- OrderTest
- UserTest

resources

This folder contains JSON schema files used for schema validation.

Store Module

In the Store module, I have covered:

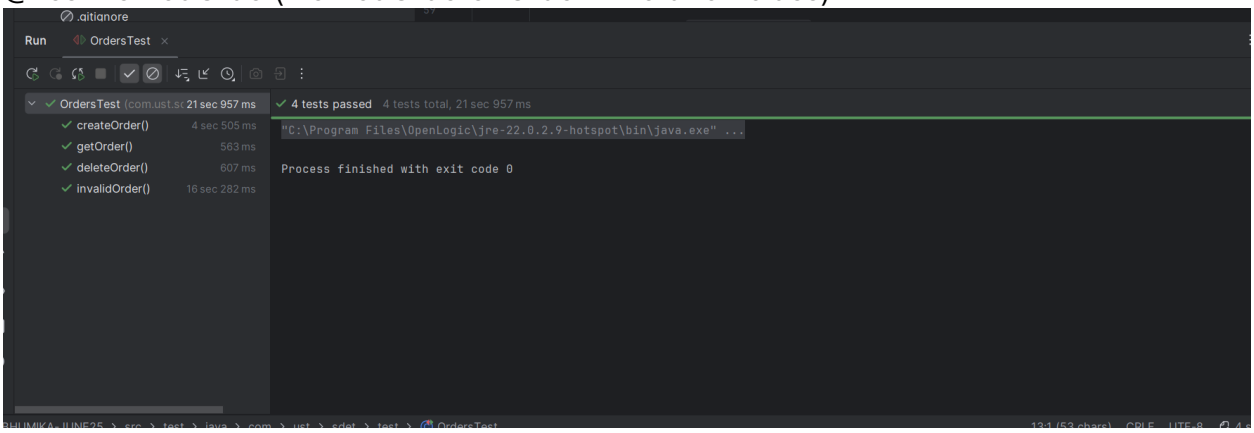
- Create Order
- Get Order Details
- Delete Order
- Invalid Order Validation

For order APIs, the flow followed is:

Create Order → Get Order → Delete Order

So here order api is depending upon createorder So that's why am using here

@TestMethodOrder(MethodOrderer.OrderAnnotation.class)



User Module

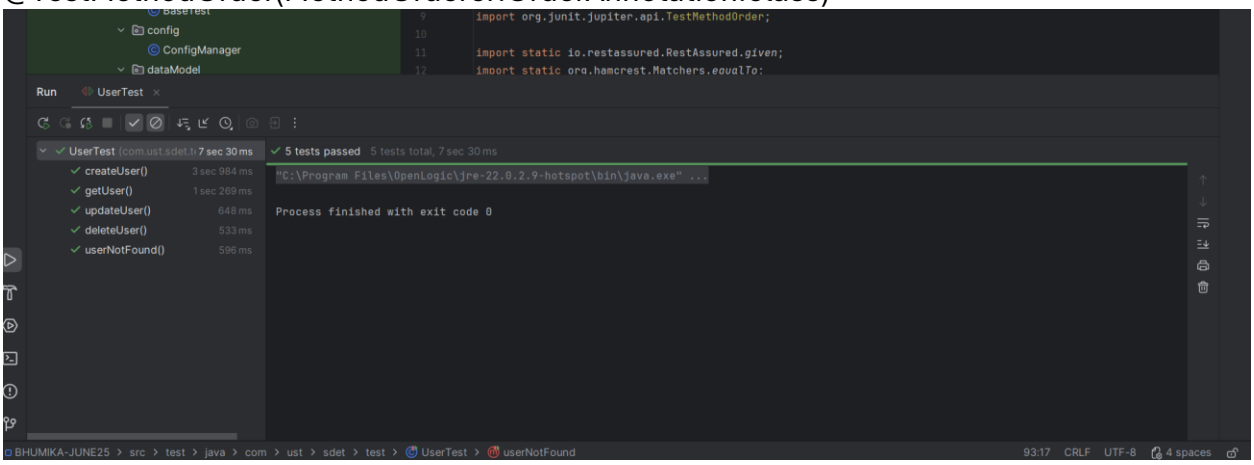
The User APIs are dependent on user creation. Therefore, the following flow is used:

Create User → Get User → Update User → Delete User

The following scenarios are covered:

- Create User
- Get User
- Update User
- Delete User
- Invalid User Validation

  here also I have used same strategy
  @TestMethodOrder(MethodOrderer.OrderAnnotation.class)



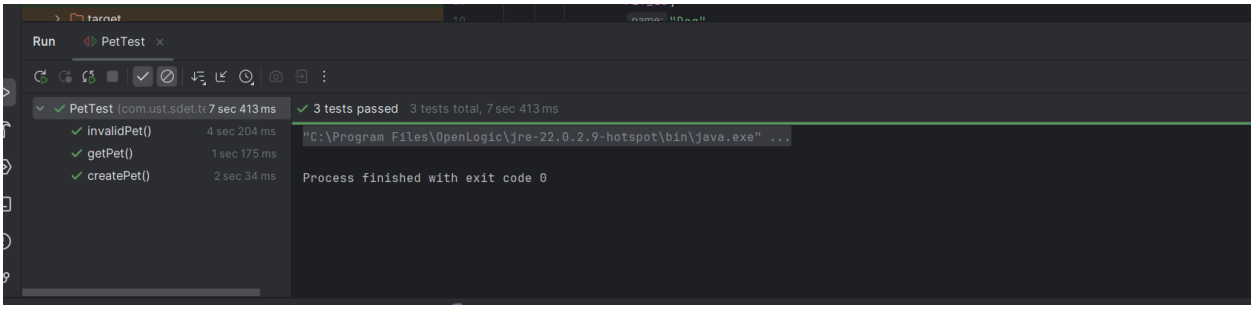
Pet Module

The Pet APIs the following flow is used:

Create pet→ Get pet by id

The following scenarios are covered:

- Create Pet
- Get Pet by ID



Schema Validation

Schema validation is implemented for the GET APIs.

The following schema is used:

- order_schema.json

Conclusion

This framework covers positive, negative, and schema validation scenarios. The code is organized in separate packages, which makes it easy to maintain.